UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH
Centre de la Imatge i la Tecnologia Multimèdia

Lab Session 6: Multiplayer Game

Engine requirements

Networking Class

- void Start()
- void onPacketReceived(byte[] inputPacket, Socket fromAddress)
- void OnUpdate()
- void onConnectionReset(Socket fromAddress)
- void sendPacket(byte[] outputPacket, Socket toAddress)
- void onDisconnect()
- void reportError()
- Clients and server will have the same UDP interface, the difference will be how they use it

Client

- Uses Networking interface
- Connects to Server
- Sends Hello packet
- Receives welcome
- Accumulates and sends periodically input data (e.g. controller)

Server

- Uses Networking interface
- Creates a socket
- Receives Hello Packets
- Creates player data
- Sends welcome packets
- Receives input packets
- Updates players with received inputs

ClientProxy Struct: Relevant server-side data from each client

Methods for:

- SpawnPlayer() -> creates a new clientProxy with all the relevant data
- SpawnGO() -> creates

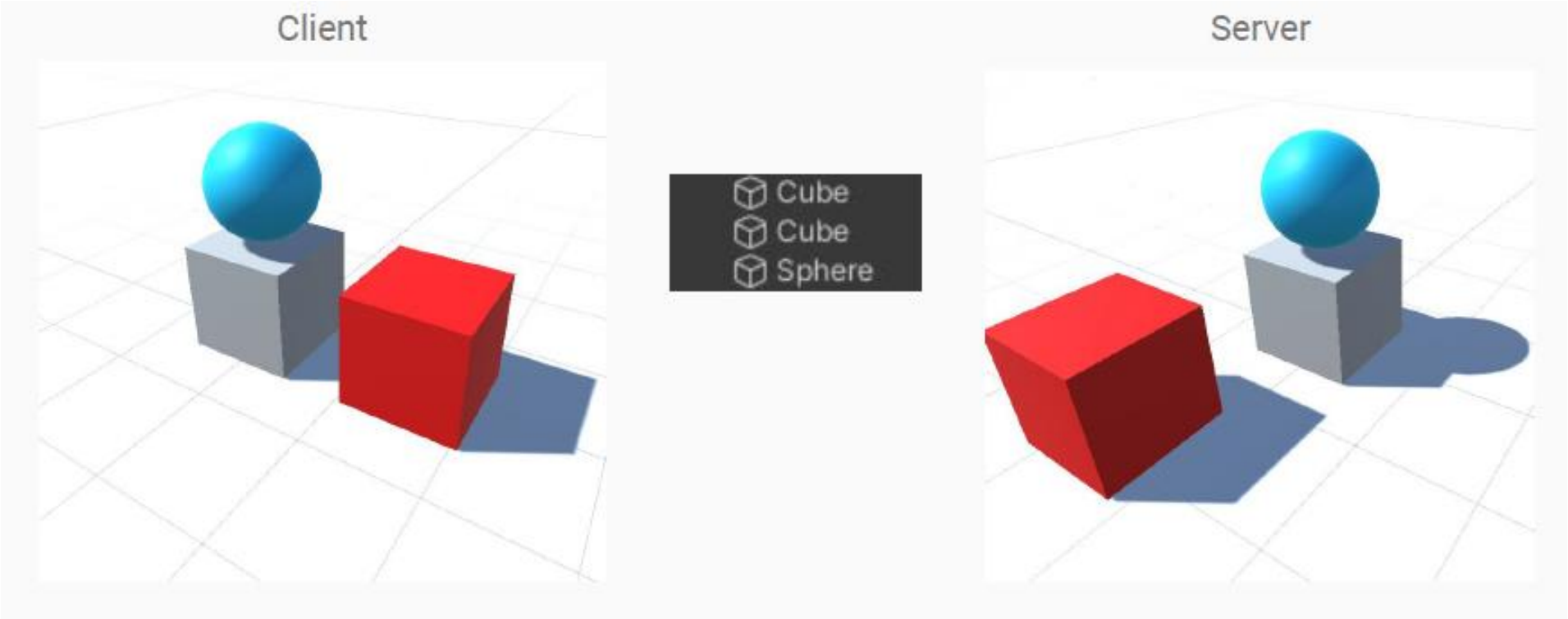
UI Layer

- Section Communications
- # Packets sent
- # Packets received
- # Networked Objects
- Disconnect (Button)
- Section Real World conditions
- Simulate Latency / Jitter (check box)
- Simulate packet drops (check box)

UI Layer

- Section Server (only on server)
  - Ping interval
  - Disconnection Timeout
  - Client Data
    - Address, port, name, ID last packet time, seconds since reply
  - Render Colliders
- Section Client (only on client side)
  - Player Info (Id, name)
  - GO Info (type, network ID, coordinates, ..)

Warning!



Netwrok Ids

- Unique IDs to identify objects over the network
  - Every client knows what objects are changing
    - Not all objects have a network ID:
    - Have NetID
      - Player
      - Bullets
      - etc.
    - No NetID
      - Static environment elements
      - UI
      - objects with predictable movements (careful!)

Linking Objects

- Register ID: Create a new ID (server side) vs. use a specific ID (client side)
- Find objects by ID (both in network and application sides)
- Unregister ID: Remove object from network IDs

Virtual UDP Connection

UDP vs TCP

- UDP is not TCP:
  - Not connection oriented
  - No knowledge of when the client is “alive”
- How to mitigate?
  - Ping packets and timeouts

Client Side

- Every defined time period:
  - Send a Ping message to Server
  - Receive Ping message from Server and save last time ping was received
  - If (currentTime – timeLastPing) > timeOut: disconnectFromServer()

Server Side

- Every defined time period:
  - Send Ping to all clients every time interval
  - Receive ping from every client and save timeout on the respective clientProxies
  - Disconnect and destroy clients whose (currentTime – timeLastPing) > timeOut